

HACKING WITH SWIFT



SWIFTUI GUIDE BOOK

Tips, tricks, milestones,
and exercises to help
you learn faster

Paul Hudson

Hacking with iOS SwiftUI Guide Book

Paul Hudson

Contents

Introduction	5
Hello.	
Milestone 1: The Swift programming language	7
What you learned	
Key points	
Challenge	
Milestone 2: Projects 1-3	15
What you learned	
Key points	
Challenge	
Milestone 3: Projects 4-6	28
What you learned	
Key points	
Challenge	
Milestone 4: Projects 7-9	39
What you learned	
Key points	
Challenge	
Milestone 5: Projects 10-12	48
What you learned	
Key points	
Challenge	
Milestone 6: Projects 13-15	62
What you learned	
Key points	
Challenge	

Milestone 7: Projects 16-18	72
What you learned	
Key points	
Challenge	
Afterword	80

Introduction

How to use this guide book

Hello.

Hacking with iOS is a complete, standalone tutorial series that teaches Swift and iOS at the same time, using real-world apps and real-world problems. I wrote it to be the course I wish I had when I was learning: practical and hands-on, full of things to learn in every chapter, but also just enough repetition to ensure new concepts had time to sink in.

Important: This guide is for the SwiftUI edition of Hacking with iOS. If you're following the Classic edition of Hacking with iOS you should follow the Classic guide book.

While Hacking with iOS is both complete and standalone, it is ultimately just a series of hand-picked iOS projects sorted by increasing difficulty. It *doesn't* make any attempt to help you monitor your progress, and so it's easy to rush through the projects, copy what I've written, but feel like you're getting nowhere – feel like you would be unable to produce your own, original apps.

That's where this guide book comes in: I've written it to act as a companion book to Hacking with iOS. There are 7 milestones in total, and each milestone contains a summary of what you've learned so far, along with a few refreshers on key parts of the code, and a challenge – a task you should try and do by yourself in order to help your new knowledge sink in. On occasion I provide example solutions to these challenges, but mostly they are entirely down to you.

So: go ahead and read the Hacking with iOS introduction chapter now, which gives you a primer on the Swift programming language. Once you're done, come back here and read the milestone. This chapter is quite long, so rather than put it inside both the Classic and SwiftUI editions of the book you'll only find it in the Classic edition.

Good luck! And remember: don't spend all your time sharpening your pencil when you should be drawing.

Milestone 1

The Swift
programming
language

What you learned

Swift is technically a C-like language, but as you’ve seen it has a number of features that really make it stand out from the pack. Please don’t worry if you read this chapter and thought, “you know, that just makes no sense” – that’s okay. In fact, it’s perfect normal. Some features, not least optionals, take a good long time to sink in, and you’ll have lots more practice with them as the series progresses.

This was a big chapter with lots of ground covered, so your head might be spinning a bit. If you find something in the list below that you don’t remember, just flick back and re-read. Don’t worry, though: all these things will be covered again in future tutorials, so you’ll have more than enough chance to learn!

- Xcode’s playgrounds are a great way to test run your code and see immediate results. We don’t use them much in Hacking with Swift because we’re building real projects, but you’ll find them invaluable in your own learning.
- Swift coders like to make things constant using **let** where possible, rather than variable using **var**. Xcode even warns you if you get this wrong.
- There are lots of types of data, such as strings (“Hello”), integers (5), doubles (5.5555), and booleans (true).
- You can tell Swift exactly what data type a value holds, but it’s preferable to let Swift figure it out from your code.
- You can perform arithmetic using +, -, *, and /, and compare with >, <, and ==.
- String interpolation lets you place values into strings. For example, “**Your name is \ (name)**” will put the value of **name** directly into the string.
- Arrays let you group lots of values together, and are type safe – that means you can’t add a number to a string array. You access items in an array by referring to their position, e.g. **names[4]**. These positions count from zero.
- Dictionaries also let you group values together, but you get to specify a key rather than just using numbers. For example, **person["month"]**.
- Conditional statements, e.g. **if person == "hater"**, let you take some action depending on the state of your program. You can combine multiple conditions together using **&&**.

- Swift has several types of loops. You saw **for i in 1...10**, which counts from 1 up to 10 inclusive, **for song in songs**, which loops every item in the **songs** array, and also **while true**, which causes the loop to go around forever until you exit.
- You can use **continue** to stop the current run of the loop and continue from the next item – it effectively jumps back to the top of the loop and carries on.
- You can use **break** to exit a loop entirely.
- Using **switch/case** blocks is a good way to execute lots of different code depending on a value. You can use ranges, e.g. **2...3**, but you must always cover every possible value. You can provide a **default** case to mean “anything not already matched.”
- Functions let you write chunks of re-usable code that you can call from elsewhere in your program. They can accept parameters, and can return values.
- Any data type in Swift can be optional, which means it might not have a value. For example, a variable holding someone’s age would be 18 if that person was 18, or 0 if that person were only a few weeks old. But what if you didn’t know their age? That’s where optionality comes in – you can set the number to **nil**, meaning “no value yet.” (This is the most confusing thing in Swift, and takes some getting used to.)
- Swift forces you to unwrap optionals before you use them, to avoid the chance of you accessing non-existent data. This is done using **if let**.
- Optionals have a special variety called implicitly unwrapped optionals, which can also have no value but don’t need to be unwrapped before you use them. If you try this and get it wrong, your code will crash – be careful!
- Optional chaining lets you safely write complex instructions using optionals. For example **thingA?.thingB?.thingC = "meh"** will do nothing if either **thingA**, **thingB**, or **thingC** are **nil**.
- The nil coalescing operator, **??**, lets you provide a default value if an optional is empty.
- Enumerations – or just enums for short – let you declare a new data type that contains a set of specific values, such as **rainy**, **cloudy**, and **sunny**. Swift lets you add extra values to its enums, to track things like “how cloudy is it?”
- Structs and classes are complex data types: they hold multiple values inside them (called “properties”), and can also have internal functions (called “methods”) that operate on those internal values.

Milestone 1: The Swift programming language

- Classes are more powerful than structs because one class can inherit from (i.e. build upon) another. However, that adds extra complexity, which isn't always helpful.
- You can attach **willSet** and **didSet** observers to properties, which means you can ask for code to be run when that property changes.
- You can make attach access control to properties, which stops other people touching them directly. This is helpful because it forces other developers to use methods to read and write values, which means you can attach extra functionality as needed.
- If you know you're working with an object of type **A** and Swift *thinks* you're working with an object of type **B**, you can use **as?** typecasting to tell Swift what it really has. You can use **as!** instead, but again your app will crash if you're wrong.
- Closures are a bit like functions that you can place inside variables and pass around. They capture any values that they need to work.

Phew! That's a lot, but again: all those things will be covered again in more detail in upcoming tutorials – this was just a primer.

Key points

I hope you can recognize that Swift takes code safety *seriously*. This takes a number of forms, of which the most important ones are:

1. All your variables and constants must have a specific type, like **Int** or **String**. There is a special **Any** type, used when you want to mix data together, but even then you'll need to typecast values back to **Int** or whatever in order to use them.
2. If you create a function that says it will return a value, Swift will ensure it always does, no matter what route is taken through its code.
3. Optionals ensure you always work with real values. Swift won't let you touch optionals if they are nil, unless you specifically force override it – there's a reason **!** is sometimes called “the crash operator”.
4. When you use a **switch/case** block, it must be exhaustive – it must cover all possible cases.

Let's take another look at some optional code. Consider this:

```
var a: Int? = 5
var b: Int? = 10
```

That creates two variables, **a** and **b**, both of which store optional integers. That means they might contain a number like 0, 10, 2309832, or they also be nil – they might have no value at all. How would you add those two numbers together?

Remember: Swift doesn't let you use optionals directly; you need to unwrap them safely first. This is because they can be nil, and “5 + nil” is undefined behavior that used to crash your app or produce weird behavior before Swift came along with its optionals.

In this case, we can unwrap **a** and **b** using **if let**. This creates new constants that are regular integers rather than optional, so we can work with them directly:

```
if let unwrappedA = a {
    if let unwrappedB = b {
```

Milestone 1: The Swift programming language

```
    let c = unwrappedA + unwrappedB
  }
}
```

That **let c** line will only be reached if both **a** and **b** had a value, so Swift knows it's safe to run.

Swift lets you merge those two **if let** statements into one, like this:

```
if let unwrappedA = a, let unwrappedB = b {
    let c = unwrappedA + unwrappedB
}
```

While that's common practice, I prefer to avoid it in this book because it's a bit harder to read.

Another important take-away is that Swift is a *modern* programming language. It used many of the latest development techniques when it launched, as has since evolved further. This means we benefit from great features like optionals, closures, **switch/case** ranges, and more.

Finally, I hope you feel like you're able to go back to the playground now and start writing a little of your own code. Sure, there's no iOS code yet, but you can still have a go at attempting the challenge below.

Challenge

At this point you should be fairly comfortable with data types, conditions, loops, functions, and more, so your first challenge is to complete the Fizz Buzz test. This is a famous test commonly used to root out bad programmers during job interviews, and it goes like this:

- Write a function that accepts an integer as input and returns a string.
- If the integer is evenly divisible by 3 the function should return the string “Fizz”.
- If the integer is evenly divisible by 5 the function should return “Buzz”.
- If the integer is evenly divisible by 3 and 5 the function should return “Fizz Buzz”
- For all other numbers the function should just return the input number.

To solve this challenge you’ll need to use quite a few skills you learned in this tutorial:

1. Write a function called **fizzbuzz()**. It should accept an **Int** parameter, and return a **String**.
2. You’ll need to use **if** and **else if** conditions to check the input number.
3. You need use modulus, **%**, to check for even division. You’ll also need to use **&&** to check for two things at once, because “Fizz Buzz” should only be printed if the input number is evenly divisible by 3 *and* 5.

Here are some test cases for you to use:

```
fizzbuzz(number: 3)
fizzbuzz(number: 5)
fizzbuzz(number: 15)
fizzbuzz(number: 16)
```

When your code is complete, that code should produce “Fizz”, “Buzz”, “Fizz Buzz” and “16”.

Please try to code your answer now. I’ve written my own solution below to get you started, but it will help you internalize Swift better if you try writing your own code first. Remember: there is no learning without struggle, so you should just try – don’t worry if you have a hard time or don’t manage to finish.

Milestone 1: The Swift programming language

If you're finding it hard, here are some hints. I suggest you read these only if you really need them, and even then only read one at a time – you should try to solve the challenge while reading as few hints as possible!

- Your function should start with **func fizzbuzz(number: Int) -> String {**. That is, it should accept an integer and return a string.
- You return values using the **return** keyword, e.g. **return "Fizz"**.
- Checking that a number is evenly divisible by 3 is done using **if number % 3 == 0**.
- Check that it's evenly divisible by 3 and 5 is done using **if number % 3 == 0 && number % 5 == 0**.

OK, that's it for hints. Below is my solution:

```
func fizzbuzz(number: Int) -> String {  
    if number % 3 == 0 && number % 5 == 0 {  
        return "Fizz Buzz"  
    } else if number % 3 == 0 {  
        return "Fizz"  
    } else if number % 5 == 0 {  
        return "Buzz"  
    } else {  
        return String(number)  
    }  
}
```

Milestone 2

Projects 1-3

What you learned

You've made your first two SwiftUI projects now, and completed a technique project too – this same cadence of two apps and a technique project continues to the end of the course, and will help you advance your knowledge quickly while taking the time to go back and refine what you learned.

Although we are only three projects into SwiftUI, you've already learned some of the most important concepts: views, modifiers, state, stack layouts, and more – these are skills that you'll be using again and again in SwiftUI, which is why I wanted to get them down early.

Of course, you also built some real projects and completed lots of coding challenges to help solidify your learning, so hopefully you're starting to get a sense of what you know.

So far we have covered:

- Building scrolling forms that mix text with controls such as **Picker**, which SwiftUI turns into a beautiful table-based layout where new screens slide in with fresh choices.
- Creating a **NavigationStack** and giving it a title. This not only allows us to push new views onto the screen, but also lets us set a title and avoid problems with content going under the clock.
- How to use **@State** to store changing data, and why it's needed. Remember, all our SwiftUI views are structs, which means they can't be changed without something like **@State**.
- Creating two-way bindings for user interface controls such as **TextField** and **Picker**, learning how using **\$variable** lets us both read and write values.
- Using **ForEach** to create views in a loop, which allows us to make lots of views all at once.
- Building complex layouts using **VStack**, **HStack**, and **ZStack**, as well as combining them together to make grids.
- How to use colors and gradients as views, including how to give them specific frames so you can control their size.
- How to create buttons by providing some text or an image, along with a closure that should

be executed when the button is tapped.

- Creating alerts by defining the conditions under which the alert should be shown, then toggling that state from elsewhere.
- How (and why!) SwiftUI uses opaque result types (**some View**) so extensively, and why this is so closely linked to modifier order being important.
- How to use the ternary conditional operator to create conditional modifiers that apply different results depending on your program state.
- How to break up your code into small parts using view composition and custom view modifiers, which in turn allows us to build more complex programs without getting lost in code.

One thing I want you to think about is what it means to be a “view” in SwiftUI. Before you started this course you might well have thought that **Color.red** couldn’t possibly be a view, but it is. You’ve also seen how **LinearGradient** is a view, which means it’s easy to use inside our layouts.

But what about **VStack**? Or **Group**? Or **ForEach**? Are *those* views?

Yes! Those absolutely are all views in SwiftUI, which is what makes the framework so remarkably composable – we can put a **ForEach** inside a **ForEach** inside a **Group** inside a **VStack**, and it all works.

Remember, all something needs to do in order to conform to the **View** protocol is to have a single computed property called **body** that returns **some View**.

Previously we looked very closely at protocols, protocol extensions, and protocol-oriented programming in Swift, and you might have wondered why this was all so important. Well, now I hope you can see: the **View** protocol is the centerpiece of SwiftUI – anything can conform to it and start taking part in layouts, in just a few simple lines of code.

In other user interface frameworks (including Apple’s own UIKit), classes were used for this work. This meant that if you had some existing types and you wanted to use them for layout, you needed to make them inherit from **UIView** - which in turn meant getting over 200

Milestone 2: Projects 1-3

properties and methods that you probably didn't need, as well as a raft of other functionality used behind the scenes.

In SwiftUI none of that happens: we just add a single **View** conformance. This is the power of protocols and protocol extensions, and this is what makes protocol-oriented programming so important – if we add a single **body** property to our type, SwiftUI knows how to use that to do layout and rendering.

Now, if you were paying attention you might have noticed a curiosity: when we make any sort of SwiftUI view we need to make it return **some View** – we make a view, and it returns one or more other views. Those views have their own **body** properties that in turn return views, and those views have their own **body** properties that... well, you get the point.

This seems like SwiftUI has built an infinite loop for itself: if all views are made up of other views, where does it actually end?

Obviously it *does* end otherwise none of our SwiftUI code would actually work. And the trick lies in something Apple calls *primitive views* – the absolute bare building blocks of SwiftUI, which conform to **View** but return some fixed content rather than rendering some other kind of view instead.

There are quite a few of these building blocks, and they won't come as much of a surprise – **Text** is one, for example, as is **Image**, **Color**, **Spacer**, and more. Ultimately all the UI we build is created on top of these building blocks, which is what breaks the seemingly infinite loop.

Key points

There are three important points that are worth going over in more detail. This will partly be reviewing what we learned – going over it again with different examples to help make sure they are clear – but I also want to take this chance to answer some questions that are likely to have cropped up so far.

Structs vs classes

First, something that should hopefully be fairly fresh in your memory: structs and classes. These are both ways of letting us build complex data types with properties and methods, but the way they work – more specifically, how they *differ* – matters.

If you remember, there are five key differences between structs and classes:

1. Classes don't come with a memberwise initializer; structs get these by default.
2. Classes can use inheritance to build up functionality; structs cannot.
3. If you copy a class, both copies point to the same data; copies of structs are always unique.
4. Classes can have deinitializers; structs cannot.
5. You can change variable properties inside constant classes; properties inside constant structs are fixed regardless of whether the properties are constants or variables.

In Apple's original programming language, Objective-C, we used classes for pretty much *everything* – we didn't have a choice, because it was really baked into the way things worked.

In Swift we *do* have a choice, and that choice ought to be based upon those factors above. I say “ought to be” because it's not uncommon to see folks who don't care about these differences, and so use either **class** or **struct** all the time without considering the ramifications of that choice.

Whether you choose structs or classes is down to you and the specific problems you're solving. However, what I *want* you to do is consider how it conveys your *intent*. Donald Knuth said that “programs are meant to be read by humans and only incidentally for computers to

Milestone 2: Projects 1-3

execute”, which really gets to the heart of what I’m talking about: when someone reads your code, is your intent clear to them?

If you use structs most of the time, switching to a class in one particular place conveys some intent: this thing is different and needs to be used differently. If you *always* use classes, that distinction gets lost – after all, it’s extremely unlikely you need them that often.

Tip: One of the fascinating details of SwiftUI is how it completely inverts how we use structs and classes. In UIKit we would use structs for data and classes for UI, but in SwiftUI it’s completely the opposite – a good reminder of the importance of learning things, even if you think they aren’t immediately useful.

Working with ForEach

The second thing I’d like to discuss is **ForEach**, which we have used with code like this:

```
ForEach(0 ..< 100) { number in
    Text("Row \("\(number)")")
}
```

ForEach is a view just like most other things in SwiftUI, but it allows us to create other views inside a loop.

Now consider a string array like this:

```
let agents = ["Cyril", "Lana", "Pam", "Sterling"]
```

How could we loop over those and make text views?

One option is to use the same construction we already have:

```
VStack {
    ForEach(0..agents.count) {
        Text(agents[$0])
    }
}
```

```

    }
}

```

But SwiftUI offers us a second alternative: we can loop over the array directly. This takes a little more thinking, because SwiftUI wants to know how it can identify each item in the array.

Think about it: if we loop over a four-item array we will create four views, but if **body** gets re-invoked and our array now contains *five* items, SwiftUI needs to know which view is new so it can be shown in the UI. The last thing SwiftUI wants to do is throw away its whole layout and start from scratch every time a small change is made. Instead, it wants to do the least amount of work possible: it wants to leave the existing four views in place and add only the fifth.

So, we come back to how Swift can identify values in our array. When we were using a range such as `0..5` or `0..agents.count`, Swift knew for sure that each item was unique because it would use the numbers from the range – each number was used only once in the loop, so it would definitely be unique.

In our array of strings that’s no longer possible, but we can clearly see that each value is unique: the values in `["Cyril", "Lana", "Pam", "Sterling"]` don’t repeat. So, what we can do is tell SwiftUI that the strings themselves – “Cyril”, “Lana”, etc – are what can be used to identify each view in the loop uniquely.

In code, we’d write this:

```

VStack {
    ForEach(agents, id: \.self) {
        Text($0)
    }
}

```

So rather than loop over integers and use that to read into the array, we’re now reading items in the array directly – just like a **for** loop would do.

As you progress with SwiftUI we’ll look at a third way of identifying views using the

Identifiable protocol, but that will come in time.

Working with bindings

When we use controls such as **Picker** and **TextField** we create two-way bindings for them to some sort of **@State** property using **\$propertyName**. This works great for simple properties, but sometimes – only sometimes, hopefully! – you might want something more advanced: what if you want to run some logic to calculate the current value? Or what if you want to do more than just stash away a value when it's written?

If we want to react to changes in a binding, we might try to leverage Swift's **didSet** property observer, but you'd be disappointed. This is where custom bindings come in: they can be used just like **@State** bindings, except we get complete control over how they work.

Bindings aren't magic: **@State** takes away some boring boilerplate code for us, but it's perfectly possible to create and manage bindings by hand if you want to. Again, I'm not showing you this because it's common, because it's really not; my hope is that you'll need to do this rarely. Instead, I'm showing it to you because I want to eliminate the idea that SwiftUI is doing some sort of magic on your behalf.

Everything that SwiftUI does for us can be done by hand, and although it's nearly always better to rely on the automatic solution it can be really helpful to take a peek behind the scenes so you understand *what* it's doing on your behalf.

First let's look at the simplest form of custom binding, which just stores the value away in another **@State** property and reads that back:

```
struct ContentView: View {
    @State private var selection = 0

    var body: some View {
        let binding = Binding(
            get: { selection },
            set: { selection = $0 }
        )
    }
}
```

```

    )

    return VStack {
        Picker("Select a number", selection: binding) {
            ForEach(0..<3) {
                Text("Item \($0)")
            }
        }
        .pickerStyle(.segmented)
    }
}

```

So, that binding is effectively just acting as a passthrough – it doesn't store or calculate any data itself, but just acts as a shim between our UI and the underlying state value that is being manipulated.

However, notice that the picker is now made using **selection: binding** – no dollar sign required. We don't need to explicitly ask for the two-way binding here because it already *is* one.

If we wanted to, we could create a more advanced binding that does more than just pass through a single value. For example, imagine we had a form with three toggle switches: does the user agree to the terms and conditions, agree to the privacy policy, and agree to get emails about shipping.

We might represent that as three Boolean **@State** properties:

```

@State var agreedToTerms = false
@State var agreedToPrivacyPolicy = false
@State var agreedToEmails = false

```

Although the user could just toggle them all by hand, we could use a custom binding to do

Milestone 2: Projects 1-3

them all at once. This binding would be true if all three of those Booleans were true, but if it got changed then it would update them all, like this:

```
let agreedToAll = Binding(  
    get: {  
        agreedToTerms && agreedToPrivacyPolicy && agreedToEmails  
    },  
    set: {  
        agreedToTerms = $0  
        agreedToPrivacyPolicy = $0  
        agreedToEmails = $0  
    }  
)
```

So now we can create four toggle switches: one each for the individual Booleans, and one control switch that agrees or disagrees to all three at once:

```
struct ContentView: View {  
    @State private var agreedToTerms = false  
    @State private var agreedToPrivacyPolicy = false  
    @State private var agreedToEmails = false  
  
    var body: some View {  
        let agreedToAll = Binding<Bool>(  
            get: {  
                agreedToTerms && agreedToPrivacyPolicy &&  
agreedToEmails  
            },  
            set: {  
                agreedToTerms = $0  
                agreedToPrivacyPolicy = $0  
                agreedToEmails = $0  
            }  
        )  
    }  
}
```



```
)  
  
    return VStack {  
        Toggle("Agree to terms", isOn: $agreedToTerms)  
        Toggle("Agree to privacy policy", isOn:  
$agreedToPrivacyPolicy)  
        Toggle("Agree to receive shipping emails", isOn:  
$agreedToEmails)  
        Toggle("Agree to all", isOn: agreedToAll)  
    }  
}  
}
```

Again, custom bindings aren't something you'll want that often, but it's so important to take the time to look behind the curtain and understand what's going on. Even though it's incredibly smart, SwiftUI is just a tool, not magic!

Challenge

You have a basic understanding of arrays, state, views, images, text, and more, so let's put them together: your challenge is to make a brain training game that challenges players to win or lose at rock, paper, scissors.

So, very roughly:

- Each turn of the game the app will randomly pick either rock, paper, or scissors.
- Each turn the app will alternate between prompting the player to win or lose.
- The player must then tap the correct move to win or lose the game.
- If they are correct they score a point; otherwise they lose a point.
- The game ends after 10 questions, at which point their score is shown.

So, if the app chose “Rock” and “Win” the player would need to choose “Paper”, but if the app chose “Rock” and “Lose” the player would need to choose “Scissors”.

Hacking with Swift+ subscribers can get a complete video solution for this checkpoint here: **Solution to Rock, Paper, Scissors**. If you don't already subscribe, you can start a free trial today.

To solve this challenge you'll need to draw on skills you learned in tutorials 1 and 2:

1. Start with an App template, then create a property to store the three possible moves: rock, paper, and scissors.
2. You'll need to create two **@State** properties to store the app's current choice and whether the player should win or lose.
3. You can use **Int.random(in:)** to select a random move. You can use it for whether the player should win too if you want, but there's an even easier choice: **Bool.random()** is randomly true or false. After the initial value, use **toggle()** between rounds so it's always changing.
4. Create a **VStack** showing the player's score, the app's move, and whether the player should win or lose. You can use **if shouldWin** to return one of two different text views.

5. The important part is making three buttons that respond to the player's move: Rock, Paper, or Scissors.
6. Use the **font()** modifier to adjust the size of your text. If you're using emoji for the three moves, they also scale. **Tip:** You can ask for very large system fonts using **.font(.system(size: 200))** – they'll be a fixed size, but at least you can make sure they are nice and big!

I'm going to provide some hints below, but I suggest you try to complete as much of the challenge as you can before reading them.

Hints:

- Start off with the simplest logic you can: three buttons, each with logic that says “the player tapped rock, the player was trying to win, and the app chose scissors, so add 1 point.”
- Once you have that working, look for a way to simplify your logic such as an array of which items beat each move. For example, if your moves array was **["Rock", "Paper", "Scissors"]** your array of winning moves would be **["Paper", "Scissors", "Rock"]**.
- You don't need to add graphics if you don't want to; just some text views and buttons is enough. Why not try emoji?

This should be a fun exercise: there's a little bit of Swift, a little bit of SwiftUI, and a little bit of logic, all wrapped up in a game where you can really go to town on theming if you want to.

Milestone 3

Projects 4-6

What you learned

At this point you should really be starting to feel comfortable with how SwiftUI works. I know that for some people it can be a massive mental speed bump, because we lose the ability to control the exact flow of our program and instead need to construct the *entire* state then just set things in flow. However, you've now built four complete projects, and had two deep-dive technique projects, so I hope you're starting to get a feel for how SwiftUI works.

Although you now have two more apps you can work on if you want, along the way you picked up a number of valuable skills:

- How to read numbers from users with **Stepper**, including using its shorter form when your label is a simple text view.
- Letting the user enter dates using **DatePicker**, including using the **displayedComponents** parameter to control dates or times.
- Working with dates in Swift, using **Date**, **DateComponents**, and **DateFormatter**
- How to bring in machine learning to leverage the full power of modern iOS devices.
- Building scrolling tables of data using **List**, in particular how it can create rows directly from arrays of data.
- Running code when a view is shown, using **onAppear()**.
- Reading files from our app bundle by looking up their path using the **Bundle** class, including loading strings from there.
- Crashing your code with **fatalError()**, and why that might actually be a good thing.
- How to check whether a string is spelled correctly, using **UITextChecker**.
- Creating animations implicitly using the **animation()** modifier.
- Customizing animations with delays and repeats, and choosing between ease-in-ease-out vs spring animations.
- Attaching the **animation()** modifier to bindings, so we can animate changes directly from UI controls.
- Using **withAnimation()** to create explicit animations.
- Attaching multiple **animation()** modifiers to a single view so that we can control the animation stack.

Milestone 3: Projects 4-6

- Using **DragGesture()** to let the user move views around, then snapping them back to their original location.
- Using SwiftUI's built-in transitions, and creating your own.

Yes, that's a heck of a lot new information in just three projects, but because each topic has been covered in isolation ("how do lists work?") then again in the context of a real project ("let's actually use a list here"), I hope it's all sunk in. If not, don't be afraid to go back and review earlier chapters – they aren't going anywhere, and it will all help you master SwiftUI.

Before moving on, I want to add one important thing: even though you might be starting to understand how SwiftUI works, you will still find – perhaps often! – that it's hard to express exactly what you mean.

Animations are a classic example of this. With animation, we want to say "make that button – that one right there – spin around right now." And SwiftUI really isn't designed for such an imperative way of thinking: we can't just say "make the button spin."

This goes beyond the mental speed bump I mentioned earlier. You can understand how SwiftUI works, but still be drawing a blank about what it takes to make something happen. This is a particular problem for folks who have prior programming experience, because they are used to the other way of thinking – they have months, years, or perhaps even decades of muscle memory that makes it very easy to solve problems, but only if they get to dictate exactly how everything should behave.

Remember, in SwiftUI all our views – and all our animations – must be a function of our state. That means we don't tell the button to spin, but instead attach the button's spin amount to some state, then modify that state as appropriate. Often this becomes frustrating, because we know where we want to get to but *don't* know how to get there.

If this hits you during the course, just relax – it's normal, and you're not alone. If you've fought with something for an hour or two and can't quite get it right, put it to one side and try the next project, then come back in a week or so. You'll know more, and have had more practice, plus a fresh mind never hurts.

What you learned

Key points

There are three things I want to discuss in more detail before we move on. Again, this is really just to make sure you've fully understood certain key concepts before we move on, and I hope it helps give you a sense of how Swift and SwiftUI are actually *working* underneath.

Ranges with ForEach and List

As I've said several times, when we create views in a loop SwiftUI needs to understand how to identify each item uniquely so that it can animate data coming and going. This in itself isn't complex, but there's a particular usage that throws people off, and that's *ranges*.

First, let's look at some code:

```
ForEach( 0..<5) {  
    Text( "Row \b($0)" )  
}
```

That loops from 0 to 5, printing out some text each time. SwiftUI can be sure that every item is unique because it's counting over a range, and ranges don't have duplicated values.

In fact, if you look at the SwiftUI code behind our **ForEach** you'll see it's actually this:

```
public init(_ data: Range<Int>, @ViewBuilder content: @escaping  
(Int) -> Content)
```

The view builder – the thing that actually assembles our views – will be given one integer from the range, and is expected to send back some view content that can be rendered.

Now try writing this:

```
ForEach( 0...5) {  
    Text( "Row \b($0)" )  
}
```


That counts from 0 *through* 5, meaning that it will create six views. Or at least it *would* create six views if it actually worked – that code doesn’t compile.

Look again at the type of data **ForEach** wants: **Range<Int>**. That’s a range of integers, but it’s a very specific range – there’s a second, very similar type called **ClosedRange<Int>**, and that’s what’s causing the problem.

When we write **0..**5**** we get a **Range<Int>**, but when we write **0...**5**** we get a **ClosedRange<Int>**. Even though it looks similar to us, Swift considers these two range types to be different and so we can’t use a closed range with **ForEach** – it just isn’t possible right now, although I hope that will change.

What makes a string?

From our perspective, it’s easy to imagine a string as being a fairly trivial thing: there’s one letter, then another letter, then a third, a fourth, and so on, perhaps with some punctuation scattered in there. But in practice, strings are some of the most complex features in Swift, and it’s worth taking a minute to understand what’s happening.

First, you might have noticed that code such as this isn’t allowed:

```
let name = "Paul"  
let firstLetter = name[0]
```

That attempts to read the first letter of the string “Paul”. If we asked a human to “run” that code, they’d say “P”, which makes sense because that’s the first letter.

However, in practice strings are much more complicated than single letters: many emoji are made up of several characters back to back to describe what they contain. For example a simple thumbs up emoji comes in a variety of skin colors, which is achieved through a base emoji (thumbs up) then a skin tone modifier (light through dark). That ends up being multiple individual characters, but we see a single one: a thumbs up emoji with a particular color.

Milestone 3: Projects 4-6

If Swift treated those characters individually, then reading the first letter would read the thumbs up emoji without a skin tone, and reading the second letter would read the skin tone without the thumbs up – the former would be acceptable but not quite as the sender expected, and the latter would just be weird.

Now think about code like this:

```
print(name.count)
```

That will print out how many characters are in our test string, which again seems easy enough. But as we've just seen, some individual characters are actually supposed to be grouped together to create a combined meaning, which means **count** can't just return how many characters the string has. Instead, it needs to start from the first letter and count through every unique letter (taking into account all the modifiers that join together) to produce a total.

It's not fast, but it's guaranteed to be correct. If nothing else, that's what I want you to take away from strings: they can be a bit tricky sometimes, but Swift is doing a huge amount of work on our behalf to make sure we don't make mistakes by accident. This does mean more code from us when using simple strings, but it also means we get automatic support for advanced strings – including any emoji you can think of – in the future.

Flat app bundles

In Word Scramble we looked for `start.txt` in our bundle, then loaded it in for use in the game. I explained then that all iOS, macOS, tvOS, and watchOS apps ship as bundles that combine their binary file (the compiled Swift program), their `Info.plist`, their asset catalog, and more.

One thing I *didn't* mention is how those bundles get built, and in particular I want to mention asset catalogs and loose files.

First, asset catalogs are where we've been storing images that we want to use in our app, and they are more than just a fancy way of organizing pictures. In fact, when Xcode builds our asset catalog it goes through all our pictures and optimizes them for iOS devices, then puts the result into a compiled asset catalog that can be loaded efficiently. As you progress further with

asset catalogs you'll learn that they can handle vector assets, colors, textures, and much more – they are versatile things!

Second, loose assets are for all the *other* kinds of media in our app – text files, JSON, XML, movies, and more. If you have lots of these files you can make groups inside Xcode to organize them, but at build time that all goes away: all those files get put into a single directory called your *resource directory*. This happens so that when we ask the bundle to find the URL for “start.txt” it doesn't need to search through all the directories in our app bundle, but instead can look in just one place because all the files are there.

This creates an interesting problem, and it's one you're guaranteed to hit sooner or later: because all your loose files from everywhere in your Xcode project end up getting placed in one resource directory, you can't use the same asset filename twice anywhere in your project. It doesn't matter what groups the files are in, or how far they seem apart in your Xcode project: if you have two files called start.txt in your project, your build will fail because Xcode can't put them both into the same directory.

Challenge

Before we proceed onto more complex projects, it's important you have lots of time to stop and use what you already have. So, today you have a new project to make entirely by yourself, with no help from me other than some hints below. Are you ready?

Your goal is to build an “edutainment” app for kids to help them practice multiplication tables – “what is 7 x 8?” and so on. Edutainment apps are educational at their core, but ideally have enough playfulness about them to make kids want to play.

Breaking it down:

- The player needs to select which multiplication tables they want to practice. This could be pressing buttons, or it could be an “Up to...” stepper, going from 2 to 12.
- The player should be able to select how many questions they want to be asked: 5, 10, or 20.
- You should randomly generate as many questions as they asked for, within the difficulty range they asked for.

If you want to go fully down the “education” route then this is going to be some steppers, a text field and a couple of buttons. I would suggest that's a good place to start, just to make sure you have the basics covered.

Once you have that, it's down to you how far you want to take the app down the “entertainment” route – you could throw away fixed controls like **Stepper** entirely if you wanted, and instead rely on colorful buttons to get the same result.

This is one of those challenges that is best approached step by step: get something working first, then improve it as far as you want. Maybe you're happy with a simple app, or maybe you really want to spend some time crafting a fun design. It's down to you!

Important: It's really easy to get sucked into these challenges and spend hours fighting with a particular bug that only exists because you wanted to get an exact effect. Don't overload yourself with work, because you'll just burn out! Instead, start with the simplest possible code that works, then build up slowly.

If you have lots of time on your hands, you could use something like **Kenney's Animal Pack** (which is public domain, by the way!) to add a fun theme to make it into a real game. Don't be afraid to add some animations, too – it needs to appeal to kids 9 and under, so bright and colorful is a good idea!

To solve this challenge you'll need to draw on skills you learned in all the projects so far, but if you start small and work your way forward you stand the best chance of success. At its core this isn't a complicated app, so get the basics right and expand only if you have the time.

At the very least, you should:

1. Start with an App template, then add some state to determine whether the game is active or whether you're asking for settings.
2. Generate a range of questions based on the user's settings.
3. Show the player how many questions they got correct at the end of the game, then offer to let them play again.

Once you have your code working, try and see if you can break up your layouts into new SwiftUI views rather than putting everything in **ContentView**. This requires passing data between views, which isn't something we've covered in detail yet, so in the meantime send data using closures – the button action from your settings view would call a function passed in by the parent view that starts the game with the user's settings, for example.

I'm going to provide some hints below, but I suggest you try to complete as much of the challenge as you can before reading them.

Hints:

- You should generate all your questions as soon as your game starts, storing them as an array of questions.
- Those questions should probably be their own Swift struct, **Question**, storing the text and the answer.
- When it comes to asking questions, use another state property called **questionNumber** or

Milestone 3: Projects 4-6

similar, which is an integer pointing at some position in your question array.

- You can get user input either using buttons on the screen, like a calculator, or using a number pad text field – whichever you prefer.
- If you intend to pass a closure into a view’s initializer for later use, Xcode will force you to mark it as **@escaping**. This means “will be used outside of the current method.”

At its simplest, this is not a hard app to build. Get that core right – get the fundamental logic of what you’re trying to do – then think about how to bring it to life. Yes, I know that part is the fun part, but ultimately this app needs to be *useful*, and it’s better to get the core working than try for everything at once and find you get bored part-way through.

Milestone 4

Projects 7-9

What you learned

Hopefully you felt like these projects started to stretch you a little, not only in pushing your SwiftUI skills further but also teaching you some more advanced Swift. Plus, of course, you also have two new SwiftUI projects that you built – you can go ahead and keep customizing these all you want, put them on GitHub, or perhaps convert them into something else that better suits your tastes.

Here’s a quick recap of all the new things we covered in the last three projects:

- Why **@State** works with structs.
- How to use **@Observable** to store data in classes.
- Presenting and dismissing views using the **sheet()** modifier and the **dismiss** environment key.
- Using **onDelete(perform:)** to enable swipe to delete.
- Adding **EditButton** to navigation bar items, to let users edit list data more easily.
- Reading and writing data with **UserDefaults**.
- Archiving and unarchiving data with **Codable**, including working with data stored in a hierarchy.
- Using the **Identifiable** protocol to make sure all items can be identified uniquely in our user interface.
- How to use **containerRelativeFrame()** to make content fit the screen.
- Using **ScrollView** to lay out custom views in a scrollable area.
- Using Swift’s generics system to write methods that work with different kinds of data.
- Pushing new views onto the navigation stack using **NavigationLink**.
- Navigating programmatically, including with the type-erased **NavigationPath**.
- How to customize the navigation bar appearance.
- Placing toolbar items in exact locations.

I hope you’ll agree that’s a *lot*, and it also spans a wide range – we’ve gone from hard-core language features through to user-facing views, with a huge spread in between. Some folks will prefer the pure language stuff and others will prefer the more creative side of coding, and

What you learned

that's OK – we all learn differently!

Key points

Although we covered a lot in the previous three projects, there are three specific things I want to get into in more detail.

Classes vs structs: what's the difference and why does it matter?

Swift gives us two ways to create our own complex data types, and it's important you understand why we have them and which to choose for any given task.

The fundamental difference between a class and a struct is that one is a *value* type and the other is a *reference* type. These are standard programming terms for how we work with data: is the data just a simple value such as “Hello” or 5, or is it a merely a signpost saying “my data is stored in RAM at this location.”

Once you understand that difference, structs and classes become two very different things, but when you're *learning* those differences can feel not very different at all. Think of it like this: when we make a variable that holds a struct, that data is literally stored inside the variable. In comparison, when we use a *class*, that data is put in memory somewhere and the variable holds a long number that identifies the location of that memory.

This is where the name comes from: “reference types” are stored as a reference to some memory somewhere, a bit like a signpost. Rather than a variable pointing directly to my house, it points to a signpost that points to my house – there's an extra layer of indirection. This is why if you point two or more variables to the same instance of a class they can modify the same data: you just have several signposts all pointing to the same house.

This is *also* why reference types and value types behave differently when used as constants. If we make a constant instance of a class, what we've done is make a constant signpost – we've said “this signpost always points to house number 24601, and can't point to a different house.” However, that doesn't stop us from remodeling the house: maybe we want to add an extra floor, or change the kitchen, or perhaps even demolish the house entirely and build a new one. If you want those things to be fixed – if you want the actual house itself to be constant – then

you need to use constant properties for your class.

So, we can make a constant signpost (**let myHouse = House()**) while having variable data (**var numberOfFloors = 3**). But we can *also* flip things around: we can make a variable signpost (**var myHouse = House()**) that has constant data (**let numberOfFloors = 3**), and that behaves very differently: we can move the signpost around so that it points to different houses, but we can't remodel the houses themselves.

Now think about how all this relates to Swift, SwiftUI, and even UIKit. If you have three screens in an app, all of which share the same data, it's important to make sure that data stays synchronized both behind the scenes (all the variables contain the same values), and to the user (all our lists / text views / etc show the same values).

SwiftUI provides wrappers such as **@State** to make sure our views stay updated as their data changes, however these were *not* available to use with UIKit – you needed to respond to changes yourself, then update the user interface to reflect those changes.

This created a problem:

- View A could create an instance of a class.
- View A could pass that to View B so they are sharing it.
- View B could then change the data and update its UI.
- View A would have no idea the data changed, and would show the old UI.

As a result of this, it was common for UIKit developers to use structs for data, because it meant that every view had its own copy of the data and it couldn't change by surprise. Even more interestingly, all of UIKit's view types were built using classes, which meant UIKit developers build their views as classes and use structs for their data – the complete opposite of SwiftUI.

Using UserDefaults wisely

UserDefaults lets us store small amounts of data easily – it's automatically attached to our app, which means it's there ready to load as soon as our app launches. While it's very useful

Milestone 4: Projects 7-9

(and you'll be relying on it heavily!) it does have two drawbacks:

1. You should only store small amounts of data there – anything over about 512KB is dubious.
2. You can only store certain types of data easily; everything else must use **Codable** first to get some binary data.

The list of types that are supported by **UserDefaults** is short and precise: strings, numbers, dates, URLs, and binary data, plus arrays and dictionaries of those types. Excluding URLs (which are really just fancy strings), all those are the same types that can be stored in a plist file – short for a *property list*.

This isn't a coincidence: **UserDefaults** actually writes out its data using a property list just like our Info.plist file. In fact, keeping this link in mind can really help you make the best of **UserDefaults** – it would be strange if our Info.plist file contained 100,000 entries of data, and it's just as strange to put 100,000 items in **UserDefaults**.

So, use the **UserDefaults** system for what it was designed for – as Apple's own documentation says, it's called user defaults "because they're commonly used to determine an app's default state at startup or the way it acts by default."

When to use generics

We used generics to create a decoding method that is capable of taking any JSON file from an app bundle and loading into a **Codable** type of our choosing. But – and this is a big but! – we first wrote the method to be *non-generic*: if you recall, it originally decoded an array of astronauts before being upgraded to load any kind of **Codable** type.

That wasn't me just wasting your time, but instead introducing you to a sensible approach to thinking about generics and protocols. In this project we needed to decode an array of **Astronaut** instances from astronauts.json, so we wrote a method to do precisely that – no protocols and no generics, just a simple extension on **Bundle** to help keep the code organized. This mimics the way our brain thinks: we can understand concrete things like astronauts, and we can describe them pretty easily.

Things aren't quite so straightforward with protocols and generics, though – we now have a range of possible types to work with, which might be entirely unrelated apart from conforming to the same protocol. For example, integers and strings conform to Swift's built-in **Comparable** protocol, which is why Swift knows how to sort arrays of them, but otherwise they are completely different things.

Perhaps confusingly, we can't compare two comparable objects, and in fact even trying to return **Comparable** from a method won't work. If you don't believe me, just try it:

```
func makeString() -> Comparable {  
    "Hello"  
}
```

That won't compile, and with good reason: **Comparable** by itself doesn't mean anything. As I said, both strings and integers conform to the **Comparable** protocol, but all that means is that you can compare one integer against another, not that you can compare any **Comparable** type against another – it just wouldn't make any sense.

This is why generic *constraints* are so useful: they let us say “this can be any kind of object, as long as...” then provide some restrictions. And – perhaps counterintuitively – adding restrictions often enables more functionality. As you saw, when we said that our decoding method could work with any type, it meant we couldn't use **JSONDecoder** with it; Swift wasn't able to know it could safely decode JSON as that type until we explicitly added a **Codable** restriction.

So, the key to using generics well is not to use them at first, and when you *do* need them to add restrictions so that you get the most functionality you can.

Challenge

Before we move on to our next batch of projects, you have a fresh challenge to complete. This means building a complete app from scratch by yourself, using the skills you've acquired over the previous three projects.

This time your goal is to build a habit-tracking app, for folks who want to keep track of how much they do certain things. That might be learning a language, practicing an instrument, exercising, or whatever – they get to decide which activities they add, and track it however they want.

At the very least, this means there should be a list of all activities they want to track, plus a form to add new activities – a title and description should be enough.

For a bigger challenge, tapping one of the activities should show a detail screen with the description. For a tough challenge – see the hints below! – make that detail screen contain how many times they have completed it, plus a button incrementing their completion count.

And if you want to make the app really useful, use **Codable** and **UserDefaults** to load and save all your data.

So, there are three levels to this app, and you can choose how far you want to go depending on how much time you have and how far you want to push yourself. I do recommend you at least give each level a try, though – every little bit of practice you get helps solidify your learning!

Hacking with Swift+ subscribers can get a complete video solution for this checkpoint here: **Solution to Habit Tracking**. If you don't already subscribe, you can start a free trial today.

Hints:

- Start with your data: define a struct that holds a single activity, and a class that holds an array of activities.
- The class will need to use the **Observable** macro so SwiftUI can monitor your data for changes.

- Your main listing and form should both be able to read the shared activities object.
- Make sure your activity conforms to **Identifiable** to avoid problems.
- Present your adding form using **sheet()**, and your activity detail view (if you add one) using **NavigationLink**. I recommend against using presentation values for your navigation – keep it simple here!

Making the button to increment completion count will challenge you, because you need to modify the activity that was passed in. If you're stuck, the easiest approach is this:

1. Make your struct conform to **Equatable**. You don't need anything special here – just add **Equatable** after **Codable** and **Identifiable**.
2. Pass both the selected activity and the **@Observable** class into your detail view.
3. When the increment button is tapped, copy the existing activity and add 1 to its completion count.
4. Use **firstIndex(of:)** to find where the previous activity was in the class's array, then change it to be your new activity – something like **data.activities[index] = newActivity** will work. (This requires the **Equatable** conformance from step 1!)

This is genuinely a useful app to build, particularly if it were specialized towards a particular interest – if the goal were practicing an instrument then you could imagine a more advanced app suggesting different things to practice, or if the goal were exercise then it might suggest new exercises to keep things mixed up.

As it is, this challenge is only a small app, but I hope it at least makes you think. Good luck!

Milestone 5

Projects 10-12

What you learned

These last three projects have really pushed hard on data, starting first with sending and receiving data using the internet, then going into SwiftData so you can see how real apps manage their data. The skills you've learned in this projects are perhaps more important than you realize, because if you put them all together you can now fetch data from the internet, store it locally, and let users filter to find the stuff they care about.

Here's a quick recap of all the new things we covered in the last three projects:

- Building custom **Codable** conformance
- Sending and receiving data using **URLSession**
- The **disabled()** modifier for views
- Building custom UI components using **@Binding**
- Adding multiple buttons to an alert
- Editing SwiftData objects using **@Bindable**.
- Using **@Query** to query SwiftData objects
- Sorting SwiftData results using **SortDescriptor**
- Filtering data using **#Predicate**
- Creating relationships between SwiftData models
- Syncing SwiftData with iCloud

That's a comparatively short list compared to some other projects, but I think it's fair to say these topics have been a real step up: SwiftData is hard in places, particularly in the way we need to work to get dynamic sorting and filtering, but it's definitely worth it!

Key points

Although we've covered a lot in these last three projects, there is one thing in particular I'd like to cover in more detail: advanced usages of **Codable**. We already looked at this a little in our projects, but it's a topic that deserves some additional time as you'll see...

Tip: If you want to know how to make SwiftData models work with **Codable**, you should read this fully.

Custom Codable keys

When we have JSON data that matches the way we've designed our types, **Codable** works perfectly. In fact, we often don't need to do anything other than add **Codable** conformance – the Swift compiler will synthesize everything we need automatically.

However, a lot of the time things aren't so straightforward, and there are three options for working with more complex data:

1. Asking Swift to convert property names automatically.
2. Creating custom property name conversions.
3. Creating completely custom encoding and decoding.

Generally speaking you should prefer them in that order, with option 1 being most preferable and option 3 being least.

Let's work through the first two, one at a time. I'll leave option 3 for the time being, because it's comparatively tricky!

Asking Swift to convert property names automatically is useful when our incoming JSON uses a different naming convention for its properties. For example, we might receive JSON property names in snake case (e.g. **first_name**) whereas our Swift code uses property names in camel case (e.g. **firstName**).

Codable is able to translate between these two as long as it knows what to expect: we need to

set a property on our decoder called **keyDecodingStrategy**.

To demonstrate this, here's a **User** struct with two properties:

```
struct User: Codable {  
    var firstName: String  
    var lastName: String  
}
```

That uses the naming convention typically used in Swift code, called camel case because the practice of uppercasing the first letters of words is a bit like humps on a camels back.

Now here is some JSON data with the same two properties:

```
let str = """  
{  
    "first_name": "Andrew",  
    "last_name": "Glouberman"  
}  
"""
```

```
let data = Data(str.utf8)
```

That JSON data uses snake case, which is a naming convention where property names are written all in lowercase with words separated by underscores.

If we try to decode that JSON into a **User** instance, it won't work because the two properties have different naming styles:

```
do {  
    let decoder = JSONDecoder()  
  
    let user = try decoder.decode(User.self, from: data)  
    print("Hi, I'm \(user.firstName) \(user.lastName)")  
}
```

Milestone 5: Projects 10-12

```
} catch {  
    print("Whoops: \(error.localizedDescription)")  
}
```

However, if we modify the key decoding strategy before we call **decode()**, we can ask Swift to convert snake case to and from camel case. So, this will succeed:

```
do {  
    let decoder = JSONDecoder()  
    decoder.keyDecodingStrategy = .convertFromSnakeCase  
  
    let user = try decoder.decode(User.self, from: data)  
    print("Hi, I'm \(user.firstName) \(user.lastName)")  
} catch {  
    print("Whoops: \(error.localizedDescription)")  
}
```

That works great when we're converting `snake_case` to and from `camelCase`, but what if our property names are completely different? This is where we need to rely on the second option: creating custom property name conversions.

As an example, take a look at this JSON:

```
let str = """  
{  
    "first": "Andrew",  
    "last": "Glouberman"  
}  
"""
```

It still has the first and last name of a user, but the property names don't match our struct at all.

When we were looking at **Codable** I said that we can create an enum of coding keys that

describe which keys should be encoded and decoded. At the time I said “this enum is conventionally called **CodingKeys**, with an S on the end, but you can call it something else if you want,” and while that’s true it’s not the whole story.

You see, the reason we conventionally use **CodingKeys** for the name is that this name has super powers: if a **CodingKeys** enum exists, Swift will automatically use it to decide how to encode and decode an object for times we don’t provide custom **Codable** implementations.

I realize that’s a lot to take in, so it’s best demonstrated with some code. Try changing the **User** struct to this:

```
struct User: Codable {  
    enum ZZZCodingKeys: CodingKey {  
        case firstName  
    }  
  
    var firstName: String  
    var lastName: String  
}
```

That code will compile just fine, because the name **ZZZCodingKeys** is meaningless to Swift – it’s just a nested enum. But if you rename the enum to just **CodingKeys** you’ll find the code no longer builds: we’re now instructing Swift to encode and decode just the **firstName** property, which means there is no initializer that handles setting the **lastName** property - and that’s not allowed.

All this matters because **CodingKeys** has a second super power: when we attach raw value strings to our properties, Swift will use those for the JSON property names. That is, the case names should match our Swift property names, and the case *values* should match the JSON property names.

So, let’s return to our example JSON:

```
let str = ""
```

```
{  
    "first": "Andrew",  
    "last": "Glouberman"  
}
```

That uses “first” and “last” for property names, whereas our **User** struct uses **firstName** and **lastName**. This is a great place where **CodingKeys** can come to the rescue: we don’t need to write a custom **Codable** conformance, because we can just add coding keys that marry up our Swift property names to the JSON property names, like this:

```
struct User: Codable {  
    enum CodingKeys: String, CodingKey {  
        case firstName = "first"  
        case lastName = "last"  
    }  
  
    var firstName: String  
    var lastName: String  
}
```

Now that we have specifically told Swift how to convert between JSON and Swift naming, we no longer need to use **keyDecodingStrategy** – just adding that enum is enough.

So, while you *do* need to know how to create custom **Codable** conformance, it’s generally best practice to do without it if these other options are possible.

Completely custom Codable implementations

So far you’ve seen how to let Swift map between snake case and camel case, and how we can specify mappings when JSON has one name and Swift uses an entirely different one.

This last option is for times when the changes are even bigger, such as if the JSON data

provides a number as a string. However, it's also useful when you want to make SwiftData models conform to **Codable**, as you'll see.

First, let's try some new JSON that demonstrates the problem:

```
let str = """
{
  "first": "Andrew",
  "last": "Glouberman",
  "age": "13"
}
"""
```

As you can see, that has unhelpful names for first name and last, but also stores a number inside a string. While there's very little we can do to fix up JSON data coming from an external server, we certainly don't want its weirdness to infect our code – that's an integer, and we want it to be stored as one in our Swift code.

So, we might define a **User** struct like this, so we correct the first and last name properties, and store **age** as an integer:

```
struct User: Codable {
    enum CodingKeys: String, CodingKey {
        case firstName = "first"
        case lastName = "last"
        case age
    }

    var firstName: String
    var lastName: String
    var age: Int
}
```

Milestone 5: Projects 10-12

But now we have a problem: while Swift can convert the property names for us, it can't handle different data types.

For this we need to create a completely custom **Codable** implementation which means adding two things to the **User** struct:

1. A new initializer that accepts a **Decoder** instance and knows how to read our properties from it.
2. A new **encode(to:)** method that accepts an **Encoder** instance and knows how to write our properties to it.

Tip: Swift uses **Decoder** and **Encoder** here because there are lots of ways of converting data to and from Swift objects – JSON is just one of several options.

Both of these take quite a bit of code, but helpfully Xcode can *sometimes* help. In this case it will actually fill in all the code required to make both these work: type **init** in the space below the properties, then press return with **init(from decoder: Decoder)** selected, then type **encode** and press return with **encode(to encoder: Encoder)** selected.

Your finished **User** struct should look like this:

```
struct User: Codable {
    enum CodingKeys: String, CodingKey {
        case firstName = "first"
        case lastName = "last"
        case age
    }

    var firstName: String
    var lastName: String
    var age: Int

    init(from decoder: Decoder) throws {
```



```

    let container = try decoder.container(keyedBy:
CodingKeys.self)
    self.firstName = try container.decode(String.self,
forKey: .firstName)
    self.lastName = try container.decode(String.self,
forKey: .lastName)
    self.age = try container.decode(Int.self, forKey: .age)
}

func encode(to encoder: Encoder) throws {
    var container = encoder.container(keyedBy: CodingKeys.self)
    try container.encode(self.firstName, forKey: .firstName)
    try container.encode(self.lastName, forKey: .lastName)
    try container.encode(self.age, forKey: .age)
}
}

```

Tip: If this were a class rather than a struct, the new initializer would need to be marked with **required** so that any subclasses are required to implement it.

That's a lot of code, but really only four lines matter: two from the initializer, and two from **encode(to:)**.

The first line that matters is this, from the initializer:

```
let container = try decoder.container(keyedBy: CodingKeys.self)
```

That uses **CodingKeys** to read all the possible keys that can be loaded from the JSON file. This looks in the **CodingKeys** enum, so we can refer to things like **.firstName** and **.age**.

The second line that matters is this, also from the initializer:

```
self.firstName = try container.decode(String.self,
forKey: .firstName)
```

That reads a string from the key **.firstName**, and assigns it to the **firstName** property of our struct. This part might be a bit confusing because we have **firstName** twice, so let me rephrase what the line of code does: "look in the JSON to find the property matching **CodingKeys.firstName**, and assign it to our local **firstName** value."

This little dance matters, because **CodingKeys.firstName** isn't actually called **firstName** because we renamed it to match our JSON. So, in practice this line *actually* means "find the **first** property in the JSON and assign it to **firstName** in our struct" – it makes sure all the automatic renaming still happens.

If it helps, imagine reading the code like this:

```
self.structFirstName = try container.decode(String.self,  
forKey: .jsonFirstName)
```

That's the first two interesting lines of code. The *second* two are effectively inverses of the first two. These are both in the **encode(to:)** method:

```
var container = encoder.container(keyedBy: CodingKeys.self)  
try container.encode(self.firstName, forKey: .firstName)
```

That first line means we want to create a place where we can store all our **CodingKeys** values, and the second one writes the current **firstName** property to whatever is specified in **CodingKeys.firstName** – again, that's important so we get the automatic renaming to **first**.

At this point, chances are you're wondering how you'll ever remember this code, because it's not exactly something you can guess. So, here's my #1 tip:

When you need to implement a custom **Codable implementation and Xcode can't generate it for you, just create a new, simple struct with one property and a one-case **CodingKeys** enum, have Xcode generate *that*, then use its implementation to help you build your own for the bigger type.**

This is particularly important when working with SwiftData, where adding **Codable** support means create a custom implementation. It's annoying to have to remember all the code above, and Xcode almost certainly won't help, so just create a temporary struct that Xcode *can* generate a **Codable** implementation for, then use its structure to make your SwiftData model class **Codable**.

Anyway, we got to this point because we were trying to load a string into an integer, which means making two changes to the code Xcode generated for us.

First, this line of code needs to change:

```
self.age = try container.decode(Int.self, forKey: .age)
```

That attempts to read the **age** property as an integer, which will fail. Instead, we need to read it as a string, then convert that to an integer or provide a default value if conversion fails.

Replace the code with this:

```
let stringAge = try container.decode(String.self, forKey: .age)
self.age = Int(stringAge) ?? 0
```

The second thing that needs to change is in **encode(to:)**, so if we need to write any JSON we keep the existing format. Here, this line needs to change:

```
try container.encode(self.age, forKey: .age)
```

That writes an integer, but it needs to write a string like this:

```
try container.encode(String(self.age), forKey: .age)
```

I know creating the custom implementation seems like a lot of hassle, but as you can see it gives us exact control over what happens: we can add any kind of logic to our loading and saving, changing names, changing types, providing default values, and more.

Challenge

It's time for you to build an app from scratch, and it's a particularly expansive challenge today: your job is to use **URLSession** to download some JSON from the internet, use **Codable** to convert it to Swift types, then use **NavigationStack**, **List**, and more to display it to the user.

Your first step should be to examine the JSON. The URL you want to use is this: **https://www.hackingwithswift.com/samples/friendface.json** – that's a massive collection of randomly generated data for example users.

As you can see, there is an array of people, and each person has an ID, name, age, email address, and more. They also have an array of tag strings, and an array of friends, where each friend has a name and ID.

How far you implement this is down to you, but at the very least you should:

- Fetch the data and parse it into **User** and **Friend** structs.
- Display a list of users with a little information about them, such as their name and whether they are active right now.
- Create a detail view shown when a user is tapped, presenting more information about them, including the names of their friends.
- Before you start your download, check that your **User** array is empty so that you don't keep starting the download every time the view is shown.

If you're not sure where to begin, start by designing your types: build a **User** struct with properties for **name**, **age**, **company**, and so on, then a **Friend** struct with **id** and **name**. After that, move onto some **URLSession** code to fetch the data and decode it into your types.

You might notice that the date each user registered has a very specific format: 2015-11-10T01:47:18-00:00. This is known as ISO-8601, and is so common that there's a built-in **dateDecodingStrategy** called **.iso8601** that decodes it automatically.

While you're building this, I want you to keep one thing in mind: this kind of app is the bread and butter of iOS app development – if you can nail this with confidence, you're well on your

way to a full-time job as an app developer.

Tip: As always, the best way to solve this challenge is to keep it simple – write as little code as you can to solve the challenge, and for you to feel comfortable that it works well.

Hacking with Swift+ subscribers can get a complete video solution for this checkpoint here:

Solution to Friendface, part 1. If you don't already subscribe, you can start a free trial today.

Milestone 6

Projects 13-15

What you learned

These projects began introducing you to the more difficult parts of SwiftUI, although none of it was really SwiftUI's fault – it's the places where SwiftUI meets Apple's older frameworks that things get a little rough. Over time these rough edges will get smoothed off a little, but it might be a few years and regardless these will still be important for any times you want to integrate code from outside of Apple.

While completing the three projects, you also learned:

- How property wrappers become structs
- Presenting confirmation dialogs with lots of buttons
- Using Core Image to manipulate images
- Importing photos into SwiftUI
- Integrating maps into SwiftUI
- Dropping pins on maps
- Sharing content from your app
- Adding **Comparable** conformance to custom types
- Finding and writing to the user's documents directory
- Enabling file encryption when writing
- Authenticating users with Touch ID and Face ID

Each of those topics were covered as a standalone technique and then applied in a real project, so hopefully it's really sinking in!

Key points

There are two topics I want to dive into more today, both of which are quite advanced but both are things that I know will really help deepen your understanding of what was covered in the previous three projects.

Operator overloading

When we added a custom conformance to **Comparable** we need to add one method called `<`. This in turn allows Swift to compare expressions such as `a < b`, which is what gave us access to a version of `sorted()` that took no parameters.

This is called *operator overloading*, and it's what allows us to add two integers or join two strings using the same `+` operator. You can define your own operators if you want, but it's also easy to extend existing operators to do new things.

As an example, we could add some extensions to **Int** that lets us multiply an **Int** and a **Double** – something that Swift doesn't allow by default, which can be annoying:

```
extension Int {  
    static func *(lhs: Int, rhs: Double) -> Double {  
        return Double(lhs) * rhs  
    }  
}
```

Pay particular attention to the parameters: it uses an **Int** as the left-hand operand and a **Double** as the right-hand operand, which means it *won't* work if you swap those two around. So, if you want to be complete – if you want either order to work – you need to define the method twice.

However, if you want to be *really* complete then extending **Int** is the wrong choice: we should go up to a protocol that wraps **Int** as well as other integer types such as the **Int16** we used with Core Data. Swift puts all integer types into a single protocol called **BinaryInteger**, and if we write an extension on that then **Self** (with a capital S) refers to whatever specific type is being

used. So, if it's used on an **Int** then **Self** means **Int**, and if it's used on an **Int16** then it means that instead.

Here's an extension that adds ***** between any kind of integer and **Double**, regardless of whether the integer is on the left or on the right:

```
extension BinaryInteger {
    static func *(lhs: Self, rhs: Double) -> Double {
        return Double(lhs) * rhs
    }

    static func *(lhs: Double, rhs: Self) -> Double {
        return lhs * Double(rhs)
    }
}
```

If you were wondering, there is a reason Swift doesn't enable these operators for us: it's not guaranteed to be as accurate as you might hope. As a simple example, try this:

```
let exampleInt: Int64 = 50_000_000_000_000_001
print(exampleInt)

let result = exampleInt * 1.0
print(String(format: "%.0f", result))
```

That creates a 64-bit integer holding 50 quadrillion and one. It then multiplies it by the **Double** 1.0 using our custom extension, which in theory means the number that comes back should be identical to the **Int**. However, that **String(format:_:)** call asks for the number to be printed with no decimal places, and you'll see it isn't the same as the integer: it is 50 quadrillion without the 1. Now, you might ask "what's 1 when you're working with 50 quadrillion", and that's fine – I'm not here to tell you what's right and wrong, only to say that if you want absolute accuracy you should avoid using these kinds of helper methods.

Milestone 6: Projects 13-15

More generally, I want to give you a warning about operator overloading. When I introduced it back in project 14 I said operator overloading “can be both a blessing and a curse,” and I want to touch briefly on why that is.

Consider code like this:

```
let paul = User()  
let swift = Language()  
  
let result = paul + swift
```

What type of data is **result**? I can think of several possibilities:

1. Maybe it’s another **User** object, now modified so that it has Swift in its array of known languages.
2. Maybe it’s a **Programmer** object combining a user and a language.
3. Maybe it’s a weird remake of the classic horror movie The Fly.

The point is that we don’t know, and we can’t really tell without reading into the source code for the relevant `+` operator.

Now consider this code:

```
let paul = User()  
let swift = Language()  
paul.learn(swift)
```

This is much clearer, I think: we’re now running a simple method on an object, and you’d probably guess that we’re mutating **paul** to include Swift in the array of programming languages.

Any good developer will tell you that clarity is one of the most important features of well-written code – we need to make our meaning clear in what we write, because it will be read dozens or hundreds of times in the future.

So, by all means add operator overloading to your arsenal of skills that you can deploy to solve problems, and indeed I go into way more detail about them in my book **Pro Swift**, but always use them with care.

Custom property wrappers

You've seen that property wrappers are really just some sleight of hand: they take a simple value and wrap it in another another value so that some extra functionality can be added. This might be the way SwiftUI uses **@State** to store values elsewhere, or how it uses **@Environment** to read values from a shared data source, but the principle is the same: it takes a simple value and gives it super-powers somehow.

We can use property wrappers in our own code, and there are lots of reasons why you might want to do so. As with operator overloading you'll understand more about how things work if you try them out, but it's also worth using them thoughtfully: if they are the first thing you reach for you're probably making a mistake.

To demonstrate property wrappers, I want to start with a simple struct that wraps some kind of **BinaryInteger** value. We're going to give this thing some custom code when it comes to setting its wrapped value, so that if the new value is below 0 we instead make it exactly 0 so that this struct can never be negative.

Our code would look like this:

```
struct NonNegative<Value: BinaryInteger> {
    var value: Value

    init(wrappedValue: Value) {
        if wrappedValue < 0 {
            value = 0
        } else {
            value = wrappedValue
        }
    }
}
```

Milestone 6: Projects 13-15

```
}

var wrappedValue: Value {
    get { value }
    set {
        if newValue < 0 {
            value = 0
        } else {
            value = newValue
        }
    }
}
}
```

We can now create that with an integer, but if that integer drops below 0 then it will be clamped to 0. So, this will print 0:

```
var example = NonNegative(wrappedValue: 5)
example.wrappedValue -= 10
print(example.wrappedValue)
```

What property wrappers let us do is use that for any kind of property in a struct or class. Even better, it only takes one step: writing **@propertyWrapper** before the **NonNegative** struct, like this:

```
@propertyWrapper
struct NonNegative<Value: BinaryInteger> {
```

That's it – we now have our own property wrapper!

In case you hadn't guessed from their name, property wrappers can only be used on *properties* rather than plain variables or constants, so to try ours out we're going to put it inside a **User** struct like this:

```
struct User {  
    @NonNegative var score = 0  
}
```

And now we can create a user and add or remove points freely, knowing for sure the score will never go below 0:

```
var user = User()  
user.score += 10  
print(user.score)  
  
user.score -= 20  
print(user.score)
```

As you can see, there really is no magic here: not only are property wrappers just syntactic sugar that causes one piece of data to be wrapped around another, but we can also make them ourselves if needed.

Challenge

Have you ever been to a conference or a meetup, chatted to someone new, then realized seconds after you walk away that you've already forgotten their name? You're not alone, and the app you're building today will help solve that problem and others like it.

Your goal is to build an app that asks users to import a picture from their photo library, then attach a name to whatever they imported. The full collection of pictures they name should be shown in a **List**, and tapping an item in the list should show a detail screen with a larger version of the picture.

Breaking it down, you should:

- Use **PhotosPicker** to let users import a photo from their photo library.
- Detect when a new photo is imported, and immediately ask the user to name the photo.
- Save that name and photo somewhere safe.
- Show all names and photos in a list, sorted by name.
- Create a detail screen that shows a picture full size.
- Decide on a way to save all this data.

Remember to import the user's photo as **Data**, so you can write it out easily.

You can use SwiftData for this project if you want to, but it isn't required – a simple JSON file written out to the documents directory is fine, although you will need to add a custom conformance to **Comparable** to get array sorting to work.

If you *do* choose to use SwiftData, here's an important tip: when storing large data like images or movies in your models, define them using a special **@Attribute** macro like this:

```
@Attribute(.externalStorage) var photo: Data
```

That tells SwiftData not to save the image data directly inside its database, but to put it alongside instead – it's much more efficient.

Remember, you already know everything you need to make this work – good luck!

Milestone 7

Projects 16-18

What you learned

We've had some long projects recently, but that's mainly a result of your SwiftUI skills really growing – you're way past the basics now, so you're able to tackle bigger projects that solve bigger problems. I realize it can feel tiring working on these larger projects, but I hope you're able to look back on what you built and feel good – you've come such a long way!

While completing these projects, you also learned:

- Creating tabs with **TabView**.
- Using Swift's **Result** type to send back success or failure..
- Controlling image interpolation.
- Adding swipe actions to list rows..
- Placing buttons in a **ContextMenu**.
- Creating local notifications with the UserNotifications framework.
- Using third-party code with Swift package dependencies.
- How to create dynamic QR codes.
- Attaching custom gestures to a SwiftUI view.
- Using sensory feedback to make iPhones vibrate.
- Disabling user interactivity using **allowsHitTesting()**.
- Triggering events repeatedly using **Timer**.
- Tracking scene state changes as our app moves between the background and foreground.
- Supporting color blindness, reduced motion, and more.
- SwiftUI's three step layout system.
- Alignment, alignment guides, and custom alignment guides.
- Absolutely positioning views using the **position()** modifier.
- Using **GeometryReader**, **GeometryProxy**, and **containerRelativeFrame()** to make special effects.

...and you also built some real apps to put those skills into action – it's been really busy, and I hope you feel proud of what you accomplished!

Key points

Before we get on to the challenges for this project, there are two points I want to explore in more depth to make sure you've understood them thoroughly: how **map()** and **filter()** fit into the larger world of functional programming, and Swift's **Result** type.

Functional programming

Although I cover functional programming a lot in my book **Pro Swift**, I want to touch on it here too because we used it in briefly project 16 with the **map()** method. This method is designed to let us specify what we want rather than how we get there, and both are part of a wider programming approach called *functional programming*.

To demonstrate how this approach differs from a common alternative, called *imperative* programming, take a look at this code:

```
let numbers = [1, 2, 3, 4, 5]
var evens = [Int]()

for number in numbers {
    if number.isMultiple(of: 2) {
        evens.append(number)
    }
}
```

That creates an array of integers, loops over them one by one, and adds those that are multiples of two to a new array called **evens** – we need to spell out exactly how we want the process to happen.

That code is easy to read, easy to write, and works great, but we can use a functional programming method called **filter()** and rewrite it as this:

```
let numbers = [1, 2, 3, 4, 5]
let evens = numbers.filter { $0.isMultiple(of: 2) }
```

Now we *don't* need to spell out how things should happen, and instead focus on *what* we want to happen: we provide **filter()** with a test it can perform, and it does the rest. This means our code is shorter, which is awesome, but it's also improved in three other ways:

1. It's no longer possible to put a surprise **break** inside the loop – **filter()** will always process every element in the array, and this extra simplicity means we can focus on the test itself.
2. Rather than providing a closure we can call a shared function instead, which is great for code reuse.
3. The resulting **evens** array is now constant, so we can't modify it by accident afterwards.

Writing less code is always nice, but writing code that is simpler, more reusable, and less variable is even better!

Functions that accept a function as a parameter, or send back a function as their return value, are called *higher-order* functions, and both **map()** and **filter()** are examples of them. Swift has many more like them, but one of the most useful is **compactMap()**, which:

1. Runs a transformation function over every item in an array, just like **map()**.
2. Unwraps any optionals returned by that transformation function, and puts the result into a new array to be returned.
3. Any optionals that are **nil** get discarded.

So, while **map()** will create a new array containing the same number of items as the array it took in, **compactMap()** might return the same amount, fewer items, or even none at all!

To see the difference between **map()** and **compactMap()** in action, try this example:

```
let numbers = [ "1", "2", "fish", "3" ]
let evensMap = numbers.map(Int.init)
let evensCompactMap = numbers.compactMap(Int.init)
```

That creates an array of strings, then converts it to an array of integers using **map()** then

compactMap(). When that code runs, **evensMap** will contain two optional integers, then nil, then another optional integer, whereas **evensCompactMap** will contain three real integers – no optionality, and no **nil**. Much better!

Result

We used Swift's **Result** type as a simple way of returning a single value that either succeeded or failed, but there are a few important features I think you'll find useful in your own code.

First, if you think about it, **Result** is like a slightly more advanced form of optionals. Optionals either contain some sort of value – an integer, a string, etc – or they contain nothing at all, and **Result** also contains some sort of value, but now rather than nothing at all for the alternative case it must contain some sort of error.

Behind the scenes, optionals and **Result** are both implemented as a Swift enum with two cases. For optionals the enum is called **Optional** and the cases are **.none** for **nil** and **.some** with an associated value of your integer/string/etc, and for **Result** they are **.success** with an associated value or **.failure** with another associated value.

The only *real* difference between the two is that Swift has syntactic sugar in place for optionals – special syntax designed to make our life easier, because optionals are so common. So, things like **if let** and optional chaining exist for optionals, whereas **Result** doesn't have any special code around it.

Second, as you've seen a **Result** contains some sort of success value or some sort of error value, but if you ever need it there are ways of using **Result** and throwing functions interchangeably.

If you have a **Result** and want to get back to **do/catch** territory, just call the **get()** method of your **Result** – this will return the successful value if it exists, or throws its error otherwise.

As an example, consider code like this:

```
enum NetworkError: Error {
```

```

    case badURL
}

func createResult() -> Result<String, NetworkError> {
    return .failure(.badURL)
}

let result = createResult()

```

That defines some sort of error, creates a function that returns either a string or an error (but in practice always returns an error), then calls that function and puts its return value into **result**. If you wanted to use **do/catch** with that value, you could use **get()** like this:

```

do {
    let successString = try result.get()
    print(successString)
} catch {
    print("Oops! There was an error.")
}

```

To go the *other* way – to create a **Result** value from throwing code – you’ll find that **Result** has an initializer that accepts a throwing closure. If the closure returns a value successfully that gets used for the success case, otherwise the thrown error is placed into the failure case.

For example:

```

let result = Result { try String(contentsOf: someURL) }

```

In that code, **result** will be a **Result<String, Error>** – it doesn’t have a specific kind of **Error** because **String(contentsOf:)** doesn’t send one back.

The last thing you should know about **Result** is that it has functional methods you’re already used to, including **map()** and **mapError()**. For example, the **map()** method looks inside the

Milestone 7: Projects 16-18

Result, and transforms the success value into a different kind of value using a closure you specify – for example, it might transform a string into an integer. However, if it finds failure instead, it just uses that directly and ignores your transformation. Alternatively, **mapError()** transforms the *error* from one type to another, which can be helpful if you want to homogenize error types in one place.

This is one of the many things to love about functional programming: once you understand the “takes a closure and uses it to transform stuff” nature of **map()**, you’ll find it exists on arrays, **Result**, and even **Optional**!

Challenge

Your challenge this time can be easy or hard depending on how far you want to take it, but at its core the project is simple: you need to build an app that helps users roll dice then store the results they had.

At the very least you should let users roll dice, and also let them see results from previous rolls. However, if you want to push yourself further you can try one or more of the following:

1. Let the user customize the dice that are rolled: how many of them, and what type: 4-sided, 6-sided, 8-sided, 10-sided, 12-sided, 20-sided, and even 100-sided.
2. Show the total rolled on the dice.
3. Store the results using JSON or SwiftData – anywhere they are persistent.
4. Add haptic feedback when dice are rolled.
5. For a real challenge, make the value rolled by the dice flick through various possible values before settling on the final figure.

When I say “roll dice” you don’t need to create fancy 3D effects – just showing the numbers that were “rolled” is fine.

The only thing that might cause you some work is step 5: making the results flick through various values before settling on the final figure. The easiest way to tackle this is through a **Timer** that gets cancelled after a certain number of calls.

While you’re working, please take a moment to remember the accessibility of your code – try using it with VoiceOver and make sure it works as well as you can make it.

Hacking with Swift+ subscribers can get a complete video solution for this checkpoint here: **Solution to High Rollers**. If you don’t already subscribe, you can start a free trial today.

Afterword

Thank you!

You've made it to the end of this book: I hope you feel much more confident in your skills than when you started it, and I hope you also had fun too.

But while you've finished the book, you *haven't* finished learning. Programming is an art, so you shouldn't spend all your time sharpening your pencil when you should be drawing. That means you need to put what you've learned into practice, both soon and often. Remember, just joining a gym isn't enough to make you fit!

And there's more to learn from me too, whether that's more of my books, or joining Hacking with Swift+ to learn how to build the Ultimate Portfolio App, score discounts on my books, and get access to my monthly app building livestreams.

The most important thing to say, though, is thank you, even if this book is the only thing of mine you ever buy. Your support has directly contributed to me being able to share more knowledge for free on **hackingwithswift.com**, where the world learns Swift.

And so, one last time: thank you.

What next?

Level up your skills with the the most popular books from Hacking with Swift:

- Pro Swift
- Pro SwiftUI
- Swift Design Patterns
- Testing Swift
- Swift Coding Challenges

Or better yet, try my bundles to get a great selection of knowledge and also save money: the Swift Power Pack, the Swift Platform Pack, and the Swift Plus Pack can really help take your skills to the next level.

You can buy all these and more at **<https://www.hackingwithswift.com/store>**.

Hacking with Swift+

Make the ultimate investment in your iOS career, with HWS+:

- Build the Ultimate Portfolio App
- Downloadable projects and learning challenges
- 20% discount on all my books year-round
- Access to monthly app building livestreams
- Support Hacking with Swift so I can share more learning
- Free gifts and more!

Find out more and subscribe at [**https://www.hackingwithswift.com/plus**](https://www.hackingwithswift.com/plus).